[Experiment, Analysis, and Benchmark] Filtered Vector Search Under Updates: When Does an Index Beat a Scan?

Suan Lee
Semyung University, Jecheon, Republic of Korea
suanlee@semyung.ac.kr ORCID: https://orcid.org/0000-0002-3047-1167

***Abstract*—Vector databases answer nearest-neighbor queries with attribute filters over data that is continuously inserted and deleted, yet filtered search has been evaluated almost exclusively on static data. We study ten methods from eight families on SIFT, DEEP, YFCC, and 4096-dimensional arXiv collections of up to 10^7 vectors, with controlled and real predicates and three update streams, and explain the results with a cost model. Our central finding is that the point where an index overtakes an exact scan is neither a selectivity nor an admissible-set size: for every index that must check inadmissible points it grows with the collection size—as $n^{0.41}$ to $n^{0.64}$ on DEEP and $n^{0.52}$ for IVF on arXiv, as the model's $\sqrt{n}$ bound requires—so a fixed dispatch threshold is right for one n only, while label-partitioned indexes escape the growth. Under churn, IVF's exact deletions keep it accurate at two to three orders of magnitude less maintenance than the graph indexes, whereas ACORN's maximum recall collapses from 1.000 to 0.575 when deletions correlate with the attribute, because the live admissible subgraph its search walks thins out; repairing its neighbor lists on every level keeps recall at 0.997–0.998 (3 runs) for 1.10–1.15× the maintenance. On real tag predicates no index reaches recall 0.99 without a selectivity dispatcher, and inside PostgreSQL the default plan of pgvector silently returns recall as low as 0.012. A rebuilt Filtered-DiskANN pays off only above a stream-dependent ratio of queries to updates, the cost-optimal rebuild period approximately follows a square-root law, and several rankings are artifacts of the setup: 30 of 177 index-versus-scan verdicts lie within the gap between the measured points that bracket the recall target. We release all artifacts.**

***Index Terms*—vector search, approximate nearest neighbor search, filtered search, hybrid queries, dynamic indexing, benchmark**

I. Introduction

Vector databases answer nearest-neighbor queries that carry attribute predicates—"similar products *in stock*", "passages *from this tenant*"—over collections that change continuously. A large body of recent work builds indexes for one of these two requirements: filter-aware graphs that keep navigation inside a predicate [1], [2], [3], [4], range-filter structures [5], [6], [7], and streaming indexes that absorb insertions and deletions [8], [9], [10]. Experimental studies have compared filtered methods on static data [11], [12], [13], [14], [15], and benchmarks have exercised streaming updates without filters [16], [17]. A deployment, however, faces both at once, and the experimental study of Li et al. lists dynamic updates as an open problem for filtered search [12]. This paper evaluates filtered search *under updates*, against simple baselines—an exact scan, a bitmap-filtered IVF index, and a selectivity dispatcher—that are easy to deploy alongside any index.

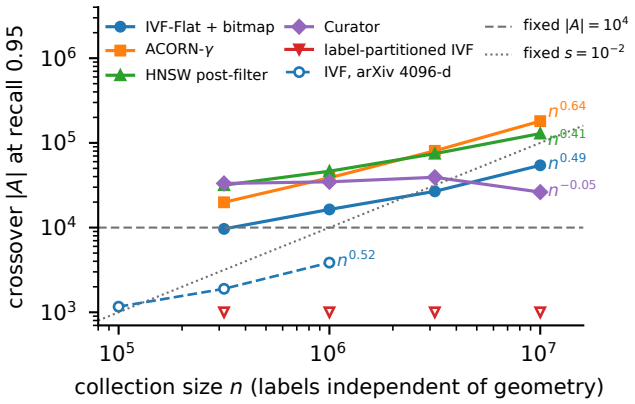


Fig. 1. Where an index catches up with an exact per-label scan, as the collection grows (recall 0.95, labels independent of geometry; DEEP subsets, and IVF on 4096-dimensional arXiv embeddings). Above its curve an index beats the scan. A dispatch threshold stated as a fixed admissible-set size (dashed) or a fixed selectivity (dotted) crosses the curves, so it is right for one n only. Exponents of log–log fits; hollow markers: the index already wins at the smallest measured size (10^3).

The crossover moves with the collection. Systems place the switch between a scan and an index with a fixed rule: pre-filter below a selectivity (Milvus: 7% [18]; ACORN [1]), or below an admissible-set size [16]. Our central finding is that neither rule transfers across collection sizes (Figure 1). An index that must check inadmissible points pays for each one it examines, so the admissible-set size at which it catches up with an exact scan grows with n: its lower bound grows as $\sqrt{n}$ in our cost model, and the measured size as $n^{0.41}$–$n^{0.64}$ for IVF, ACORN, and post-filtering in our measurements. A threshold of 10^4 admissible points, right for IVF at $n{=}3.16 \times 10^5$ (9,648), is 5.4× too low at $n{=}10^7$; a selectivity threshold of 10^{-2} is 1.8× too high there; and on 4096-dimensional embeddings, where distances cost more relative to filter checks, the same 10^4 is 2.6× too high at $n{=}10^6$. Indexes that never touch an inadmissible point escape the growth: a label-partitioned IVF beats the scan from 10^3 admissible points up at every n. Updates add a second mechanism—under correlated deletions an in-filter graph loses the subgraph its search walks—and the

rest of the paper measures both.

We ask six questions. **Q1**: When does an index beat an exact scan of the admissible points, and is the crossover a function of the admissible-set size $s \cdot n$? **Q2**: How does the correlation between attributes and geometry reorder the methods? **Q3**: What happens on real predicates whose selectivity spans orders of magnitude? **Q4**: How do accuracy, throughput, footprint, and maintenance evolve as the data churns, and what if deletions correlate with the attribute? **Q5**: Which method is cheapest once maintenance and queries are both paid for? **Q6**: Which evaluation practices change the conclusions?

Summary of findings. (1) *A crossover law* (Sections IV, V-A). From n=3.16 $\times$ 10^5 to 10^7 the crossover grows as $n^{0.49}$ for IVF, $n^{0.64}$ for ACORN, and $n^{0.41}$ for post-filtering, and not for Curator ($n^{-0.05}$); on arXiv IVF's exponent recurs ($n^{0.52}$) and post-filtering's does not. (2) Correlation reorders the methods—post-filtering is among the fastest with independent attributes and fails to reach recall 0.95 with correlated ones, a label-specialized graph the opposite—but only when queries ask for labels far from where they lie: when a query asks for its nearest neighbor's label, post-filtering is second-fastest on SIFT1M. (3) On YFCC-10M's tag predicates no index reaches recall 0.99 alone; with a tuned selectivity dispatcher every index does, and Curator for selective predicates paired with IVF or a graph for common ones is fastest, also on 500 disjoint queries. Routing on cardinalities estimated under tag independence misroutes no query, and a Roaring bitmap index makes the predicates 105$\times$ cheaper to evaluate. (4) *Percolation under correlated churn.* IVF's exact deletions keep it accurate at 119–888$\times$ less maintenance than the graph indexes. ACORN, which deletes only through its predicate, collapses (maximum recall 1.000$\rightarrow$0.575) when deletions correlate with the attribute; a label's recall tracks the effective degree of its live admissible subgraph and falls below 0.65 once that degree drops under eight, and repairing the neighbor lists around deleted nodes on every level keeps recall at 0.997–0.998 (3 runs). (5) *A rebuild law.* At recall 0.95 a freshly rebuilt Filtered-DiskANN beats IVF in total cost only above 1.3–2.2 queries per update on DEEP-5M and 17.8–38.4 on SIFT-500K, and never at recall 0.99 on SIFT-500K; with an exact delta between rebuilds the cost-optimal period follows the economic-order-quantity law, but on none of our streams does the delta design beat both IVF and rebuilding every round. (6) Rankings depend on the setup: threads raise the crossover, contention changed throughputs by up to 2.9$\times$, ACORN's documented build compiles scalar kernels, and 30 of 177 index-versus-scan verdicts lie within the gap between the measured points that bracket the recall target.

Contributions. A benchmark for filtered search under updates, with controlled selectivity and two correlations, real predicates, three update streams, and a gated protocol (Section III); a cost model with a crossover bound, a percolation account, and a rebuild-period law, tested against falsification criteria stated in advance except for the checks of Section IX (Section IV); a study of ten methods from eight families (Sections V–VII); evaluation lessons, including fixes for two ACORN defects and a repair for its deletions (Sections VI, VIII); and all artifacts, including per-query results.

II. Filtered Search and Its Update Paths

A filtered k-NN query asks for the k points nearest to q among the *admissible set* $A_p = \{x \in X : p(x)\}$ of a predicate p; its selectivity is $s = |A_p|/n$. We group the methods we evaluate into eight families by *where the predicate enters the search* and record, for each, *how it absorbs insertions and deletions* (Table I).

Pre-filter scan. Compute A_p, then scan it exactly. When the predicate is an equality on a label known at insertion time, the points of each label can be stored contiguously (*per-label partition*), so the scan touches only $|A_p|$ vectors; we also evaluate the general case in which A_p is an arbitrary bitmap and the scan visits all n rows and skips inadmissible ones (*bitmap scan*). Updates are appends and removals; there is nothing to repair.

Post-filter. Search an unfiltered index for $k' \geq k$ candidates and keep the admissible ones. We use hnswlib [19] with slot reuse for deletions. Post-filtering needs $k' \gtrsim k/s$ when admissible points are spread like the data, and fails when they are not.

Predicate-agnostic in-filter graph. ACORN-γ [1] builds an HNSW-like graph whose neighbor lists are expanded by a factor γ, so that the subgraph induced by an arbitrary predicate stays navigable; search visits only admissible nodes. Insertions use the graph's incremental `add`; ACORN has no deletion routine, so deleted points are excluded through the predicate bitmap and remain in the graph as routers. We also evaluate ACORN-1 (γ=1), its cheaper-to-build variant.

Label-specialized graph. Filtered-DiskANN [2] builds a Vamana graph whose pruning preserves, for every label, paths among the points carrying it, and starts each query from a per-label medoid. It supports equality on build-time labels only. The public Python binding exposes no dynamic filtered index, so freshness requires rebuilding.

Label-indexed tree. Curator [20] builds one hierarchical clustering tree over all vectors and embeds a lightweight per-label index in it as a subtree; a point is indexed under a label by granting it the label, and a label search traverses that label's subtree. Other predicates, such as the AND of two tags, use its bitmap-filtered search. Insertion adds the vector to the tree and grants its labels. The released implementation rejects vector removal, so deletion revokes the point's labels: the vector stays in the tree, but no label search can return it.

Partition with bitmap. IVF-Flat [21] assigns points to the nearest of n_{list} k-means centroids and scans the n_{probe} lists nearest to the query, skipping inadmissible entries via a bitmap. With centroids fixed after training, insertion is an append to one list and deletion an exact removal; there are no tombstones and no graph to repair. Correlated predicates hurt IVF in a specific way: the admissible points may live in lists far from the query, so recall at a fixed n_{probe} drops.

Label-partitioned IVF. A design point that the cost model of Section IV suggests and that we add to the study: one IVF-

TABLE I
METHOD FAMILIES, HOW EACH CONSUMES THE PREDICATE, AND HOW EACH ABSORBS UPDATES IN THE IMPLEMENTATIONS WE EVALUATE.

family	method (impl.)	predicate	insert	delete
pre-filter scan	per-label / bitmap (Faiss)	any†	append	remove
post-filter	HNSW (hnswlib)	any	add	slot reuse
in-filter graph	ACORN-γ, ACORN-1	any	add	bitmap exclusion
label graph	Filtered-DiskANN (diskannpy)	eq. label	rebuild	rebuild
label tree	Curator	label‡	add+grant	revoke label
partition+bitmap	IVF-Flat (Faiss)	any	append	remove
label partition	label-partitioned IVF (Faiss)	eq. label	append	remove
regenerable pools	RIG (ours)	any	route+append	remove

†The per-label partition requires an equality predicate on a build-time label; the bitmap scan accepts any. ‡Other predicates through a bitmap-filtered search.

Flat index per label over that label's points ($n_{\text{list}} = 4\sqrt{n_b}$ for a label with n_b points, each with its own k-means). A label search probes only that label's lists, so it never examines an inadmissible point, and insertion and deletion are exact, as in IVF. It requires an equality predicate on a build-time label.

Regenerable pool index. RIG, a design point of our own (unpublished), stores no adjacency lists: points belong to overlapping pools from hash-priority ball carving, and neighbors are recomputed at query time inside the predicate. Its state is a function of the point set, which gives exact deletions; its dynamic path routes insertions down a fixed leader tree and compacts every round.

III. BENCHMARK DESIGN

Datasets and predicates. We use SIFT1M (n=10^6, d=128), DEEP at 10^6, 3.16×10^6, and 10^7 points (d=96, normalized; the two smaller sets are fixed random subsets of DEEP-10M, so that n varies without changing the distribution), and YFCC-10M from the filtered track of the NeurIPS'23 Big-ANN competition [16] (n=10^7, d=192), whose points carry real tag sets and whose queries ask for the AND of one or two tags. On SIFT and DEEP we generate label predicates with two controlled knobs. Each point receives one of $L = 1/s$ labels, defined by quantiles of $\beta\, z(x) + (1-\beta)\,\varepsilon$, where $z(x)$ is the standardized projection $\langle x, w\rangle$ on a fixed random direction w and ε is standard Gaussian noise, so that s sets the selectivity and $\beta \in [0,1]$ controls the correlation between label and geometry (β=0: independent labels; β=1: slabs along w); each query asks for one uniformly random label. We use $s \in \{0.5, 0.1, 0.01\}$ and $\beta \in \{0, 0.5, 0.9, 1\}$. On YFCC we use 500 fixed random public queries; their selectivities range from 7.4×10^{-6} (74 admissible points) to 0.19, with 10th, 50th, and 90th percentiles of 2.2×10^{-5}, 1.3×10^{-3}, and 8.1×10^{-2}, and our exact filtered ground truth matches the official one on all of them.

Update streams. Three streams combine filtered queries (β=0.9, s=0.1) with churn. *SIFT-500K*: SIFT descriptors drawn from BIGANN (1.5×10^6 distinct vectors, 500 BIGANN queries), 5×10^5 live points, 20 rounds, each deleting 10% of the live points uniformly at random and inserting 5×10^4 unseen points (200% turnover). *DEEP-5M*: 5×10^6 live points, 10 rounds of 10% deletions and 4.99×10^5 insertions of unseen DEEP vectors (100% turnover). A third stream, *SIFT-500K-adv*, tests correlation $\times$ churn: it is SIFT-500K with the same insertions and queries, but each round deletes the 10% of live points with the largest projection on the label direction w (plus small noise), so deletions empty the geometric regions of the highest labels first and the admissible sets of those labels shrink while others grow. Exact filtered ground truth is recomputed over the live set after every round. As a control for the update paths, we also snapshot the live set of both SIFT streams after round 20 and build every method from scratch on it, with the same queries and ground truth. Every method applies every round through its own update path (Table I); search quality is measured at rounds $0, 4, \ldots, 20$ (SIFT) and $0, 5, 10$ (DEEP).

Follow-up campaign. To test the cost model we ran a second campaign under the same protocol and form every ratio within it; its experiments ran in two passes, except the instrumented and repaired ACORN streams of Section VI (one to five runs, reported as ranges), the ACORN counting build, and the DEEP-5M delta stream (one pass). It adds (i) an n-sweep with labels independent of geometry (β=0): DEEP subsets of 3.16×10^5, 10^6, 3.16×10^6, and 10^7 points, each with admissible-set sizes $|A_p| = n/L$ from about 10^3 to 3.16×10^5; (ii) query–filter correlation on SIFT1M: with probability $\rho \in \{0, 0.5, 1\}$ a query asks for the label of its exact nearest neighbor instead of a uniformly random label; (iii) per-query accounting of the work each method does—the entries IVF checks against the bitmap and the admissible ones whose distance it computes, and, in a separately compiled ACORN build, level-0 hops, predicate checks, and distance computations; (iv) instrumented ACORN streams that record, per label and round, the subgraph of live admissible nodes that search can traverse, and a local repair of that subgraph; (v) dense sweeps of every search knob in steps of $\sqrt{2}$ on the workloads whose conclusions depend on the interpolation between measured points (Section VIII); and (vi) PostgreSQL 16 with pgvector 0.8.1 (Section V-E; HNSW with m=32, ef_c=200, `ef_search` 16..1000, and IVFFlat with $4\sqrt{n}$ lists, both with iterative scans); and (vii) 4096-dimensional arXiv embeddings [14] (10^5, 3.16×10^5, and 10^6 points, 500 of the dataset's queries) with β=0 labels and, at 10^6, the real predicate "category includes the query's". The label-partitioned IVF sweeps n_{probe} from 1 to 512.

Parameters. Each method is swept over its search knob until recall exceeds 0.999 or a cap is reached: ACORN `efSearch` 16..2048, Filtered-DiskANN L 16..1024, IVF n_{probe} $8..\min(2048, n_{\text{list}}/2)$ with $n_{\text{list}} = 4\sqrt{n}$, post-filter $k' \in \{100, 400, 1600, 6400\}$ (streams: $\{400, 1600, 6400\}$), RIG ef 16..512; the same ranges apply to the streams. These sweeps, like the dispatcher thresholds of Section V-C, are evaluated on the reported queries, as is customary in ANN benchmarks; reported operating points are therefore tuned to the query set. Construction parameters follow the original papers' defaults: ACORN M=32, γ=12, M_β=64; Filtered-DiskANN R=64, L=100, α=1.2; HNSW M=32, ef_c=200.

Metrics. Every query asks for k=10 neighbors. Recall@10

of a query is the fraction of its exact filtered top-10 that is returned (over $\min(10, |A_p|)$ when fewer points are admissible), averaged over 500 queries; *maximum recall* is the highest recall in a method's parameter sweep, and QPS at a recall target is interpolated log-linearly on the method's recall–QPS Pareto frontier, except for the dispatcher of Section V-C, whose QPS is that of the best measured configuration (a concrete threshold and parameter) that reaches the target. QPS is the number of queries divided by the time spent inside the search calls of a single thread.

Measurement protocol. Searches are single-threaded and issue one query per call, with these exceptions: on label workloads ACORN is called once for all 500 queries and RIG once per label (both timed in C++), and post-filtering, the bitmap scan, and the per-label scan are called once per label (the per-label scan is also reported per query); one call per label lets BLAS share work across queries. Some method-specific costs fall on the boundary of the timer: post-filtering's selection of admissible candidates (56 μs per query at k'=6400) and Filtered-DiskANN's check of the second tag of AND queries (73 μs at L=1024) are outside it, as is the Python call into RIG's C++ kernel (354 μs, which favors RIG); ACORN's one-query-per-call mode on YFCC allocates and clears an n-entry visited table inside it (0.40 ms per call, which disfavors ACORN on YFCC). The fixed cost of a Python binding call, timed inside the measurement for the per-query methods, is 12 μs for Faiss, 21 μs for `diskannpy`, and 8 μs for hnswlib, a median 2% of the query time over all measured operating points of these methods and at most 4% at recall ≥ 0.95. The predicate is handed to every method in its native precomputed form (label, bitmap, or admissible list), so predicate evaluation is not timed for any method; on YFCC it is not negligible—intersecting a query's tag lists takes 1.4 ms on average (p99 20.2 ms), and materializing the 10^7-entry mask and packing it into a bitmap another 1.2 and 0.6 ms—the intersection is needed by every method that receives an admissible list or bitmap (including the per-query scan), the mask and bitmap only by the bitmap-filtered methods. Static builds use 32 threads; in the streams, initial builds, update maintenance, and rebuilds use 16 threads, uniformly.

Each configuration is warmed once and timed three times; the median 95% confidence interval is $\pm$0.8% of the mean (90th percentile $\pm$3.8%). The machine, a two-socket Xeon Gold 6258R (2×28 cores, 38.5 MB LLC per socket, 502 GB RAM; Ubuntu 22.04, kernel 6.8, GCC 11.4), was shared with unrelated workloads, so within-job intervals do not capture the dominant source of variance. All jobs ran pinned to one socket (memory allocated there by first touch) and strictly one at a time, and a fixed single-thread calibration kernel ran before and after every job; a job started only when the kernel reached 380 queries per second (its quiet-machine value is 500–600; 350 for the re-measurements of a later, slower session) on two consecutive checks, and was re-run if it ended below that. Every configuration was measured in two passes and we report the faster one; the second pass and every re-run ran under this gate, whereas part of the first pass predates it (and some of its jobs spanned both sockets), which the best-of-two selection mitigates but does not remove; across the 1073 configurations measured in both passes the median ratio between passes is 1.03 (90th percentile 1.10), and recall is identical across passes for the deterministic methods (the scans, IVF, RIG), differs by at most 0.003 for post-filtering, and differs by a median of 0.003 (90th percentile 0.016) for ACORN-γ, 0.001 (0.006) for Filtered-DiskANN, and 0.007 (0.042) for ACORN-1, whose multi-threaded builds are not deterministic; every reported point keeps the recall and QPS of the same run. Software: Faiss 1.15.1 (AVX2/AVX-512 dispatch), `diskannpy` 0.7.0, hnswlib 0.8.0, and ACORN at commit `c259f11` built with AVX2 and the patch of Section VIII.

IV. A Cost Model for Filtered Search

The cost of answering a filtered k-NN query decomposes into three terms:

$$T = c_{\text{nav}} \cdot H \;+\; c_{\text{chk}} \cdot E \;+\; c_{\text{dist}} \cdot D, \tag{1}$$

where H is the number of navigation steps (graph hops or centroid comparisons), E the number of candidates whose admissibility is checked, and D the number of admissible candidates whose distance is computed. The three method families differ in which terms they can avoid. The exact scan over a label partition has H=E=0 and D=$|A_p|$. A predicate-agnostic method—IVF with a bitmap, ACORN, post-filtering—learns whether a candidate is admissible only by checking it, so $E > 0$. A label-indexed structure (a per-label partition, Curator, or the label-partitioned IVF of Section II) never meets an inadmissible candidate, so E=0 and D is bounded by the admissible points near the query.

Proposition 1 (crossover lower bound). Suppose that admissibility is independent of position (the β=0 workloads) and that a predicate-agnostic method chooses the next candidate to check from positions and from the outcomes of earlier checks only, so that each newly checked candidate is admissible with probability s independently of the past. To collect k admissible candidates it checks, in expectation, at least k/s candidates, so $T \geq c_{\text{chk}}\, k/s$, while the scan costs $c_{\text{dist}}\, s\, n$. The method can beat the scan only when $s\, n \geq \sqrt{(c_{\text{chk}}/c_{\text{dist}})\, k\, n}$, that is, the crossover admissible-set size grows at least as $\sqrt{n}$:

$$m^*(n) \;\geq\; \sqrt{\tfrac{c_{\text{chk}}}{c_{\text{dist}}}\, k\, n}. \tag{2}$$

Proof sketch. Each checked candidate is admissible independently with probability s; the number of checks until the k-th admissible one is negative binomial with mean k/s (Wald's identity gives the same bound for any stopping rule that does not look ahead). Equating the two costs gives the threshold. □

The bound is loose—it ignores navigation and the extra candidates needed for recall—but it is structural: no tuning of a predicate-agnostic index removes the $\sqrt{n}$ growth, while label-indexed structures are not subject to it. Section V tests the growth exponent (H1) and whether E and D explain measured time (H2).

Percolation of the admissible subgraph. For an in-filter graph the relevant structure is the subgraph of live admissible

nodes that search can traverse, given by each node's *effective admissible out-neighbors* as the search collects them (for ACORN, direct neighbors and neighbors of neighbors beyond the first M_β list positions, until $2M$ are found). When deletions are only excluded through the predicate, deleted nodes keep their slots in these lists, and deletions concentrated on one region remove most of the live neighbors of the admissible nodes that remain there. Site percolation predicts that once the mean effective degree falls to about one, the largest component vanishes; Mandarapu and Kunkunuru derive the corresponding static boundary $s_c \approx 0.83/M$ [22]. Section VI measures effective degree and components per label and round (H3) and tests the account causally with a local repair.

Proposition 2 (rebuild period). Consider a label-specialized index rebuilt every P rounds at cost B, with points inserted since the last rebuild answered by an exact scan of a per-label delta. If a query at index age a costs $t_0 + g\,a$—g is the growth of the delta scan per round—then the cost per round with Q queries per round is $B/P + Q\,(t_0 + g\,(P-1)/2)$, minimized at

$$P^* = \sqrt{2B/(Q\,g)}, \tag{3}$$

the economic order quantity of inventory theory. Rebuilding every round (P=1) is optimal only when $Q \geq B/g$. *Proof.* Differentiate in P. □ Section VII measures $t(a)$ and compares the best period with P^* (H6).

V. Static Filtered Search

A. The exact scan is the baseline to beat (*Q1*)

An exact scan over the admissible points costs time proportional to $s \cdot n$ and nothing else; every index must beat it to be worth its build time, memory, and maintenance. To separate the admissible-set size from the collection size, Figure 2 holds $s \cdot n$ fixed at 10^4 and 10^5 and grows n from 10^6 to 10^7 (DEEP subsets, β=0.9), plotting each index's QPS relative to the per-label scan at recall 0.95 and 0.99.

The crossover grows with n for predicate-agnostic indexes. Proposition 1 predicts that m^* grows with n unless the index never examines inadmissible points. Figure 1 measures m^* with labels independent of geometry (β=0) for n from 3.16×10^5 to 10^7. At recall 0.95, m^* grows as $n^{0.49}$ for IVF (95% bootstrap interval over queries 0.47–0.52; the other exponents are point estimates, because the batched methods have no per-query times to resample), $n^{0.64}$ for ACORN, and $n^{0.41}$ for post-filtering; at n=10^7 IVF needs 54,344 admissible points to match the scan, ACORN 180,588, and post-filtering 128,836. The label-indexed structures behave the other way: Curator's m^* does not grow with n ($n^{-0.05}$), and the label-partitioned IVF, which never touches an inadmissible entry, beats the scan at every measured size from 10^3 admissible points up, at every n; at recall 0.99 its m^* is about 10^3 at n=10^7 (1,004). The bound itself is loose—with the measured cost ratio $c_{\text{chk}}/c_{\text{dist}} \approx 0.052$ it gives $m^* \geq 2,277$ at n=10^7, more than an order of magnitude below the measured values—but the separation it predicts holds: every predicate-agnostic index has a positive exponent, every label-indexed structure a non-positive one.

The cost terms account for the time. Across 236 IVF configurations, query time is 0.073 ms plus 5.58 ns per entry checked against the bitmap and 108 ns per admissible distance (R^2=0.967, median relative error 19%, 90th percentile 61%, largest for the fastest configurations, whose time is dominated by the fixed cost per call). For ACORN, distance computations alone explain little of the time (R^2=0.426); in the counting build it performs a median of 209 predicate checks per distance computation, and checks and distances together explain its time with R^2=0.980. With independent labels, the number of entries IVF checks per collected neighbor at its cheapest setting with recall $\geq$ 0.95 grows as $(1/s)^{0.517}$: above the bound's $1/s$ only through the extra lists needed for recall at large s, and approaching it as s falls.

Extrapolation. Fitted on $n \leq 3.16 \times 10^6$, a power law $T = c\,n^a s^b$ predicts throughput at n=10^7 with a median error of 11% for the label-partitioned IVF, 14% for the scan, 42% for ACORN, and 25% for IVF (ACORN reaches the target at only two of the six n=10^7 workloads); IVF's largest errors are at the lowest selectivities, where the check term k/s takes over—a change of regime that a single power law cannot express but the cost decomposition does.

A 4096-dimensional check. On arXiv (β=0, n=10^5..10^6) IVF's crossover again grows as $n^{0.52}$ (95% interval 0.51–0.53), and the label-partitioned IVF beats the scan at every measured size from 10^3 admissible points up, at every n (Curator too, except at the largest admissible sets of the two smaller collections). A distance costs more here relative to a bitmap check ($c_{\text{chk}}/c_{\text{dist}}$=0.016 against 0.052 on DEEP), and, as Proposition 1 implies and we predicted before the runs, the crossover is lower: IVF matches the scan at 3,859 admissible points on arXiv-1M, against 16,375 on DEEP-1M. Two results do not carry over: post-filtering's crossover does not grow with n on these three sizes ($n^{-0.34}$), and the cost decomposition fits IVF's time less well (median relative error 34%). On the dataset's real categories (arXiv-1M, median selectivity 0.149), only ACORN reaches recall 0.95, at 11.2× the faster exact scan (the bitmap scan: at d=4096 gathering the admissible rows costs more than scanning in place); IVF, Curator, and post-filtering stop at 0.949, 0.926, and 0.842.

With correlated labels (β=0.9, Figure 2) the same pattern appears at fixed $s \cdot n$:

At 10^4 admissible points the scan wins at every n, and by more as n grows. It answers 4,286–2,951 queries per second exactly; the best index, IVF, reaches 0.5× the scan's throughput at recall 0.95 for n=10^6 and only 0.1× for n=10^7, and every graph method fails to reach 0.95 at $n \geq 3.16 \times 10^6$. Curator, which indexes each label's points in its own subtree, is the exception to the decline: relative to the scan it stays at 0.4× for n=10^6 and 0.4× for n=10^7, and at 10^5 admissible points at 1.3× and 1.0×—the most robust index to collection size, though still slower than the scan below 10^5. ACORN's design calls for a larger expansion factor at low selectivity ($\gamma \geq 1/s$) [1]; with γ=100 on SIFT1M at s=0.01, β=0.9,

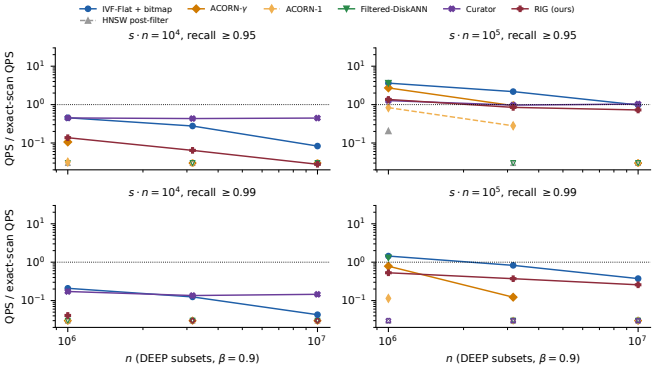


Fig. 2. QPS relative to the exact per-label scan as n grows at a *fixed* admissible-set size $s \cdot n$ (columns: 10^4, 10^5; rows: recall ≥ 0.95, ≥ 0.99; DEEP subsets, β=0.9). Above the dotted line the index beats the scan; hollow markers on the bottom edge never reach the recall target.

its maximum recall rises only from 0.946 to 0.956, while its build time grows from 181 s to 2,144 s.

At 10^5 admissible points the winner depends on n. At n=10^6 (s=0.1) IVF and Filtered-DiskANN are 3.6× and 3.6× faster than the scan at recall 0.95 (1.4× and 1.3× at 0.99); at n=3.16×10^6 only IVF still wins, by 2.2× at 0.95, and it loses at 0.99; at n=10^7 (s=0.01) IVF merely ties the scan at 0.95 (1.0×) and is 0.4× as fast at 0.99. This matches the mechanism behind Proposition 1, although its independence assumption does not hold at β=0.9: at a fixed admissible-set size, a larger collection means a lower selectivity and more inadmissible entries checked per admissible neighbor, while Curator's per-label subtree avoids them. With independent labels IVF still beats the scan at n=10^7 and $s \cdot n$=10^5 (2.1× at recall 0.95), consistent with its m^* of 54,344 there. A crossover stated as a selectivity, or even as an admissible-set size, therefore does not transfer across collection sizes.

At 10^6 admissible points indexes win. On DEEP-10M at s=0.1 the scan falls to 31 QPS; at β=0 post-filtering and IVF are 16.2× and 8.1× faster at recall 0.99, and at β=0.9 Filtered-DiskANN and IVF are 4.0× and 4.0× faster. Curator, designed for selective filters, does not reach 0.95 here even at its largest search effort (maximum recall 0.895), consistent with its authors' recommendation to pair it with a graph index for common labels [20].

When the predicate cannot be pre-partitioned, the exact scan must visit all n rows and test the bitmap; this *bitmap scan* answers 7.0 QPS on DEEP-10M versus 31 for the per-label partition, so arbitrary predicates move the crossover toward smaller admissible sets.

B. Correlation reorders the methods (*Q2*)

Figure 3 varies the label–geometry correlation β on SIFT1M at s=0.1. Post-filtering is among the fastest methods when labels are independent of geometry—854 QPS at 0.99 for β=0 on SIFT1M, within a few percent of IVF (859), and the fastest of all on DEEP-10M (506)—and the worst when they are not: its maximum recall falls to 0.903 at β=0.9 and 0.897 at

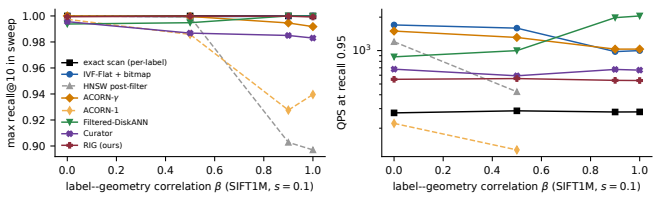


Fig. 3. Effect of label–geometry correlation on SIFT1M (s=0.1): maximum recall reached in each method's sweep (left) and QPS at recall 0.95 (right).

β=1, because a query's unfiltered neighborhood then contains few admissible points. Filtered-DiskANN moves the other way—from 290 to 913 QPS at 0.99—because its per-label medoids and label-preserving edges sit in the region where a correlated label lives. ACORN and IVF lose throughput as correlation grows (ACORN 721→243, IVF 859→489): for IVF the admissible points of a query's label live in lists far from the query, so more lists must be probed. Curator is insensitive to correlation (maximum recall 0.996 to 0.983, 680 to 667 QPS at 0.95), as a per-label index should be. On both SIFT1M and DEEP-10M, a benchmark that fixes β=0—uniformly random attributes, the common choice [12]—would thus favor post-filtering and penalize the label-specialized graph.

That penalty also depends on where queries lie. In every workload above a query asks for a uniformly random label, so with correlated labels it usually lies far from its label's region. When a query instead asks, with probability ρ, for the label of its nearest neighbor, the ranking at β=0.9 changes: post-filtering, which does not reach recall 0.95 at ρ=0, answers 2,519 QPS at ρ=1 and becomes the second-fastest method after the label-partitioned IVF; IVF gains 1.77× and ACORN 1.87×, while Curator falls to 0.601× of its throughput and Filtered-DiskANN barely changes (1.06×). With independent labels ρ matters little (IVF 1.01×, post-filtering 1.28×). A filtered benchmark therefore has to fix two correlations—of attributes with geometry and of queries with their filters—and its conclusions about post-filtering and graph indexes hold only for the second it chose.

C. Real predicates need a dispatcher (*Q3*)

YFCC-10M's queries span four orders of magnitude of selectivity, and no index reaches recall 0.99 on them by itself. Table II breaks recall down by admissible-set size. The graph methods fail on small admissible sets: ACORN's recall is 0.027 below 10^3 admissible points and 0.190 between 10^3 and 10^4, and reaches 0.995 only above 10^5; RIG, whose entry points are a 1% sample, finds no admissible entry for the most selective queries (0.460); Filtered-DiskANN, which can follow only one tag of an AND and whose rare-tag subgraphs are poorly connected (Section VIII), stays below 0.632 except on the most common tags. IVF degrades least (0.956 below 10^3). Meanwhile the per-query scan answers the smallest bucket at 5,608 QPS.

A common remedy is a selectivity-based dispatcher; ACORN's authors propose pre-filtering whenever a query's estimated selectivity is below $1/\gamma$ [1]. We evaluate the oracle version: queries whose admissible set is smaller than a threshold T are answered by the exact scan, the rest by the index, and T and the index's parameter are chosen jointly to maximize QPS at a recall target (Table II, right). With an oracle dispatcher every method reaches 0.99; IVF is fastest (311 QPS at T=32,768), ahead of ACORN (231), post-filtering (200), and RIG (141). For IVF, ACORN-γ, post-filtering, and RIG the optimal thresholds lie between 32,768 and 65,536 admissible points, the same order as the crossover of Section V-A at n=10^7; for the weaker ACORN-1 and Filtered-DiskANN they are higher (262,144 and 524,288). To check that a tuned dispatcher transfers, we drew 500 further public YFCC queries disjoint from the first set and applied to them, unchanged, the threshold and parameter chosen on the original queries. At the 0.99 target their recall is 0.984–0.994 across methods (IVF 0.994, ACORN 0.984), so the oracle's choices generalize up to a small recall loss, and the methods keep the oracle's order: IVF answers 228 QPS, ACORN 176, post-filtering 144, and RIG 113. (The held-out queries were measured in a later session on a slower machine state, so their throughputs are comparable with each other but not with the oracle column.) A deployed dispatcher routes on *estimated* cardinalities. Estimating an AND query's admissible set under tag independence underestimates it by a median factor of about 12 on YFCC, because tags co-occur; yet routing on these estimates, with the threshold and parameter tuned on exact sizes, misroutes 0 of the 500 queries for IVF. Every AND query admits at most 29,090 points, below the optimal threshold, so underestimation only moves it further to the side it belongs to, and the queries near the threshold are single-tag queries whose cardinality is known exactly. Curator has the opposite profile to the graph indexes: it is exact on selective queries (recall 1.000 below 10^3 admissible points and 1.000 between 10^3 and 10^4) and weaker on common ones (0.965 between 10^5 and 10^6). Dispatching between it and the scan therefore helps little (38 QPS at 0.99), but dispatching between it and a graph or IVF—its authors' recommended use [20]—is the fastest configuration we found: Curator below T=65,536 admissible points and IVF above reach 0.99 at 368 QPS, and Curator with ACORN at 346, ahead of the best scan-based dispatcher (IVF, 311); on the disjoint held-out queries the two pairings keep their lead (312 and 315 QPS at recall 0.989 and 0.985, versus 228 for scan plus IVF). An index's YFCC result without a dispatcher is therefore dominated by its failures on selective queries, which a dispatcher routes to the scan.

D. Build cost and memory

The methods differ by up to 41× in build time and 10× in memory (Table III). IVF builds DEEP-10M in 60 s at the size of the data; ACORN needs 2,109 s and Filtered-DiskANN 1,331 s on the same 32 threads, and Filtered-DiskANN's build is per label schema, so a new attribute means a new index. ACORN-1, the variant without neighbor expansion, builds 22–23× faster than ACORN-γ (94 s on DEEP-10M) but trades recall for it: its maximum recall at s=0.1, β=0.9 is 0.928 on SIFT1M and 0.967 on DEEP-10M, versus 0.995 and 0.985 for ACORN-γ.

TABLE II

YFCC-10M (REAL TAGS). LEFT: RECALL BY ADMISSIBLE-SET SIZE $|A|$ AT EACH METHOD'S HIGHEST-RECALL SETTING (QUERIES PER BUCKET IN PARENTHESES). RIGHT: QPS AT RECALL ≥ 0.99 WITH A DISPATCHER THAT SCANS QUERIES WITH $|A| < T$, ORACLE (TUNED ON THESE QUERIES) AND HELD-OUT (THE SAME CHOICE ON 500 DISJOINT QUERIES, A LATER SESSION; RECALL IN PARENTHESES), AND THE HELD-OUT P99 LATENCY IN MS.

	recall by $\lvert A\rvert$					dispatched, recall ≥ 0.99		
	$<10^3$ (123)	10^3–10^4 (112)	10^4–10^5 (113)	10^5–10^6 (116)	$\geq 10^6$ (36)	oracle QPS	held-out queries QPS (rec.)	p99 ms
IVF	0.956	0.998	0.999	1.000	1.000	311	228 (0.994)	14.2
ACORN-γ	0.027	0.190	0.777	0.995	1.000	231	176 (0.984)	30.4
ACORN-1	0.015	0.189	0.748	0.980	0.989	82	62 (0.989)	121.1
post-filter	0.435	0.721	0.932	0.997	0.997	200	144 (0.993)	30.4
F-DiskANN	0.093	0.179	0.122	0.632	0.975	37	33 (0.989)	261.3
Curator	1.000	1.000	0.998	0.965	0.953	38	34 (0.988)	261.3
RIG	0.460	0.895	0.991	0.991	0.950	141	113 (0.992)	43.3
scan (per query)	1.000	1.000	1.000	1.000	1.000	11	—	—

TABLE III

BUILD TIME (S, 32 THREADS) AND INDEX SIZE (GB) AT $n = 10^7$.

	DEEP-10M		YFCC-10M	
	build	GB	build	GB
exact scan	—	3.9	—	7.7
IVF-Flat	60	3.9	80	7.8
HNSW (post-filter)	457	6.6	1,013	10.4
ACORN-γ	2,109	8.9	3,273	12.8
ACORN-1	94	7.2	148	11.0
Filtered-DiskANN*	1,331	10.3	1,844	19.5
Curator§	229	4.2	392	18.7
RIG	181	39.6	333	73.1

*Per label schema. §Size measured as resident-memory growth; per label schema; on YFCC only the queried tags are indexed (int16 label ids).

E. Inside a database system

The same questions arise inside a DBMS, where the optimizer, not the user, picks the access path. We loaded SIFT1M and the DEEP-1M n-sweep workloads into PostgreSQL 16 with pgvector 0.8.1 and issued each filtered query as SQL (`WHERE label = ℓ ORDER BY emb <-> q LIMIT 10`), one backend per query. With its default settings the optimizer chooses the HNSW index for 9 of 10 workloads without iterative scans, and the query then returns a recall between 0.012 and 0.939: the index yields the vector's nearest neighbors, the filter discards most of them, and the result is silently short. Only for the most selective schema ($|A_p|$=10^3) does it pick the exact plan (B-tree on the label, then sort), with recall 1.00. Forced into correct plans, the crossover reappears: HNSW with iterative scans overtakes the exact plan at about 13,275 admissible points on DEEP-1M (IVFFlat at 12,625), near IVF's crossover in Faiss at the same n (16,375). In absolute terms the DBMS is far slower: on DEEP-1M at s=0.1 its exact plan answers about 36× fewer queries per second than the per-label scan in Faiss and IVFFlat about 15× fewer than IVF (measured in different sessions, so

only the order of magnitude is meaningful). Amanbayev et al. observed that pgvector's optimizer picks approximate index scans where an exact scan is as fast [23], and Lu et al. that filter checks dominate graph search inside PostgreSQL [15]; we quantify the recall the plan choice loses. Correlation again reorders the methods: on SIFT1M at s=0.1 HNSW falls from 290 to 22.7 QPS with correlated labels. Milvus 2.6 [18] (standalone, same workloads and parameters) has a dispatcher built in, and it is a fixed rule: its HNSW search falls back to brute force whenever at least 93% of a segment is filtered out, i.e. below a selectivity of 7% at any n. On DEEP-1M this brute force, a bitmap pass over the segment, answers 70.9–97.7 QPS at $s \leq 3.1\%$, while IVF_FLAT in the same server reaches recall 0.95 at 163–186 QPS there, 1.67–2.49× faster down to 10^3 admissible points, so at this n the threshold is too high. Its default search parameters give recall 0.860–0.906 on the graph path, and a search takes 3.78 ms even on a one-row collection (median).

F. Multi-threaded throughput

All throughputs so far are single-threaded. We repeated three workloads with 28 search threads on the physical cores of one socket, in each method's better mode (its native batched call, or 28 threads issuing single-query calls; batched won wherever both ran, and query-parallel Faiss calls at 10^7 points abort inside the OpenBLAS bundled with `faiss-cpu`), the better of two passes, and against single-thread runs from the same session. ACORN scales 16.1–17.4×, post-filtering 16.7×, IVF 10.9–13.7×, RIG 5.9–7.2×, and the memory-bound scan only 5.4–10.1×; the Filtered-DiskANN binding has no batched filtered search, and a thread pool of its per-query calls is slower than one thread. Rankings change with the workload: post-filtering is fastest at 28 threads on DEEP-10M with β=0 (14,760 QPS at recall 0.95), IVF stays ahead of ACORN with β=0.9, and ACORN overtakes IVF on SIFT1M (15,489 versus 11,399). Because the scan scales least, threads raise the crossover: at n=10^7 with independent labels, IVF's m^* at recall 0.95 grows from 54,344 with one thread to 143,307 with 28 and ACORN's from 180,588 to 234,677, while post-filtering's falls (128,836 to 122,392) and the label-partitioned IVF still beats the scan at 10^3 admissible points. Which method is fastest therefore depends on whether queries are served one at a time or in batches across all cores; the update-stream comparisons that follow are single-threaded.

VI. Filtered Search Under Updates (**Q4**)

Figure 5 follows each method through the three streams, and Table IV summarizes the last round. We group the methods by the update paths of Table I.

Exact deletion (scan, IVF, RIG). The per-label scan and IVF delete by removing entries, so nothing accumulates. IVF holds a maximum recall of 1.000 on SIFT-500K after 200% turnover and 1.000 on DEEP-5M after 100%, at nearly unchanged throughput (2,444→2,321 and 557→546 QPS at recall 0.95), for a total maintenance of 1.2 s and 12.4 s—the cheapest of any index (Curator needs 5–9× as much, and the graph indexes two to three orders of magnitude more). RIG also deletes exactly (maximum recall ≥0.998 throughout) at 38–39× IVF's maintenance.

Curator deletes by revoking a point's labels, which removes it from the per-label subtrees although its vector stays in the shared tree; its quality also stays flat (maximum recall 0.987 on SIFT-500K and 0.990 under correlated deletions, 0.916 on DEEP-5M, where it never reaches 0.95), at 5–9× IVF's maintenance.

Deletion through the predicate (ACORN, post-filtering). ACORN has no deletion routine; deleted points stay in the graph and are excluded by the query bitmap. Its graph therefore grows (3.0× resident nodes on SIFT-500K) and every deletion lowers the density of live admissible points that in-filter navigation must traverse. Under uniform deletions its maximum recall drifts from 1.000 to 0.993 and its throughput at recall 0.95 from 1,855 to 656; on DEEP-5M it reaches 0.99 at round 0 but not at the later measured rounds (5 and 10), and its throughput at 0.95 falls from 427 to 197, while its incremental insertions cost 119× IVF's maintenance. *Correlated* deletions are far worse: when the stream deletes the geometric region of the highest labels (SIFT-500K-adv), ACORN's maximum recall collapses from 1.000 to 0.970 after four rounds and 0.575 after twenty. A control separates the update path from the data: rebuilt from scratch on exactly the live points, labels, and queries of round 20, ACORN reaches a maximum recall of 0.995 (1,300 QPS at recall 0.95) on the correlated-deletion stream and 1.000 on the uniform one, against 0.575 and 0.993 for the incrementally maintained graph. The collapse and the drift are therefore caused by the deleted nodes that ACORN's update path keeps in the graph, not by the changed label distribution; the instrumented runs below locate the mechanism. (The fresh builds ran in a later session on a slower machine state, so their throughput is, if anything, understated.) Post-filtering with slot reuse keeps its recall under uniform churn but is slow at high recall (273 QPS at 0.95 on SIFT-500K) and capped by the correlated predicate on DEEP-5M (maximum recall 0.913); under correlated deletions it loses ground (0.990→0.964).

What collapses. Section IV predicts that in-filter search fails when the subgraph of live admissible nodes thins out. In the instrumented streams we measured, per label and measured round, the mean number of live admissible out-neighbors that ACORN's level-0 search can collect from a node (its *effective degree*) and the share of the label's live nodes in the largest connected component of that subgraph (Figure 4). A label's maximum recall tracks its effective degree: 0.997 at 16 or more, 0.969 between 8 and 16, 0.643 between 4 and 8, 0.267 between 2 and 4, and 0.061 below 2. The collapse sets in near a degree of five rather than one, and before the subgraph fragments: labels with recall below 0.5 still keep a mean share of 0.704 of their live nodes in one component. What runs out is the search's reach within its budget, not connectivity. A local repair that rewires each live node's level-0 list around its deleted neighbors (Section III) tests the mechanism: on SIFT-500K-adv it keeps maximum recall at 0.991 through round

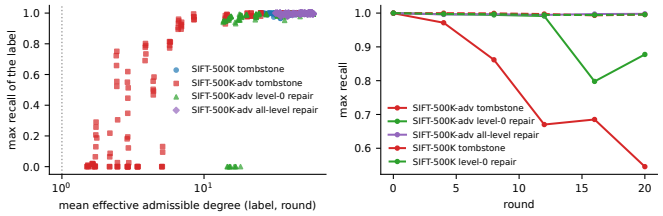


Fig. 4. ACORN under deletions excluded through the predicate. Left: maximum recall of a label's queries against the mean effective admissible degree of its live nodes (one point per label and measured round). Right: maximum recall by round with tombstones only, with a local repair of level 0, and with a repair of every level.

TABLE IV
LAST ROUND OF EACH STREAM: MAXIMUM RECALL / QPS AT RECALL 0.95 (1 THREAD; N/R: NOT REACHED) / TOTAL MAINTENANCE (S, 16 THREADS; FILTERED-DISKANN: ONE REBUILD PER ROUND).

	SIFT-500K, round 20			DEEP-5M, round 10			SIFT-500K-adv, round 20		
	rec.	QPS	maint.	rec.	QPS	maint.	rec.	QPS	maint.
exact scan	1.000	859	2.7	1.000	62	16.7	1.000	550	2.5
IVF	1.000	2,321	1.2	1.000	546	12.4	0.999	1,866	1.2
post-filter	0.991	273	246.0	0.913	n/r	1,565.4	0.964	109	208.7
ACORN-γ	0.993	656	276.1	0.982	197	1,481.1	0.575	n/r	275.1
F-DiskANN	0.999	2,490	1,043	0.996	828	8,197	1.000	3,771	1,084
Curator	0.987	885	9.7	0.916	n/r	57.5	0.990	1,498	11.1
RIG	1.000	724	46.1	0.999	202	486.5	0.999	578	46.1

12, against 0.670 without it, for 1.20–1.26× the maintenance (against tombstones in the same session), although its late-round recall varies between runs: 0.878–0.982 at round 20 over 5 runs, against 0.546–0.600 with tombstones alone. After round 12 recall falls again (0.878 at round 20), and the labels that fail have dense, connected level-0 subgraphs (mean effective degree 15.8) and still about 156 live nodes above level 0. ACORN filters its upper levels too, and this repair leaves their lists pointing at deleted nodes, so the route from the entry point into the label's region is cut. Repairing every level confirms it: maximum recall then stays at 0.995 at round 12 and 0.997–0.998 at round 20 in each of 3 runs—as high as a fresh build on the same live data—for 1.10–1.15× the maintenance of tombstones alone in the same session (the repair takes 66–67 s over 20 rounds, and insertions into the repaired graph are cheaper in every run). ACORN's collapse is thus a property of its update path that a local consolidation removes, not of in-filter search.

Rebuild (Filtered-DiskANN). Freshly rebuilt, Filtered-DiskANN is the fastest method at recall 0.95 in the last round of all three streams (2,490 QPS on SIFT-500K, 828 on DEEP-5M, 3,771 on SIFT-500K-adv; at 0.99, 948 versus IVF's 1,188 on SIFT-500K), but each rebuild takes 52 s and 820 s on 16 threads, so staying fresh every round costs 1,043 s and 8,197 s—861× and 659× IVF's maintenance. Between rebuilds, inserted points are invisible to it.

Scan throughput. The scan stays exact, and its throughput tracks the live admissible-set sizes (under correlated deletions 864→550 QPS).

VII. TOTAL COST (**Q5**)

A deployment pays for maintenance and for queries. Per round of U updates and Q filtered queries, method X costs $M_X + Q/\mathrm{QPS}_X$, where M_X is its maintenance time and QPS_X its single-thread throughput at the required recall. On all three streams IVF is cheaper than ACORN, post-filtering, Curator, and RIG on *both* terms at recall 0.95, so with single-threaded queries those methods are dominated for every ratio Q/U. With all cores serving queries the per-query term can favor ACORN (Section V-F), but its maintenance term stays 119–228× IVF's, so it pays off only for query-dominated streams—and its recall still drifts. Only a freshly rebuilt Filtered-DiskANN is faster per query; it saves 0.62 ms per query on DEEP-5M and 0.03 ms on SIFT-500K, and recovers the cost of one rebuild per round only above 1,311,450 queries per round on DEEP-5M (with 998,651 updates per round), i.e. above 1.3 queries per update (21 counting the rebuild's 16 threads as CPU time). These break-even points are unstable, because they divide a large maintenance difference by a small per-query difference: computed at each measured round they span 1.3–2.2 queries per update on DEEP-5M and 17.8–38.4 on SIFT-500K, where IVF is faster per query in 2 of 6 rounds and Filtered-DiskANN never pays off; at recall 0.99 IVF is faster per query in every round of SIFT-500K. Correlated deletions shift the balance toward Filtered-DiskANN, because IVF's throughput falls under these deletions (2,514→1,866 QPS at 0.95) while a fresh rebuild does not degrade: on SIFT-500K-adv the per-round break-even spans 1.8–207.9 queries per update at 0.95 and 0.9–2.1 at 0.99 (IVF is faster per query in 1 of 6 rounds), so in the late rounds a rebuild per round pays off with fewer queries than updates. Rebuilding less often lowers the maintenance term, and an exact per-label scan of the points inserted since the last rebuild keeps them visible (Proposition 2). Measured on the SIFT-500K stream, this design answers 2,450 QPS at recall 0.95 with a fresh index and 934 one round later, and its query time then grows by 0.298 ms per round of index age as the delta grows (rebuild: 49.7 s). The cost-optimal rebuild period approximately follows the square-root law (4 of the 5 cases with an interior optimum lie within 30% of P^*, our criterion): with 0.1 queries per update it is 6 rounds against P^*=5.77 (SIFT-500K-adv: 8 against 6.56), with 0.3 it is 3 against 3.33 (2 against 3.78); on DEEP-5M (one pass), whose rebuild takes 791 s and whose delta grows faster, it is 2 against 1.67 at 0.1; and from about one query per update on, rebuilding every round is best. On none of the three streams does the delta design beat both IVF and rebuilding every round: where a long period pays off, queries are rare and IVF's maintenance is two orders of magnitude cheaper, and where queries dominate, the delta scan dominates their cost. A faster delta would lengthen P^* as $1/\sqrt{g}$; ours is a single-threaded exact scan.

Memory follows the update path: on DEEP-5M, IVF and the scan hold 1.96 GB, Filtered-DiskANN and HNSW 5.13 and 3.63 GB, ACORN grows from 4.47 to 8.93 GB because deleted nodes stay, and RIG needs 19.83 GB.

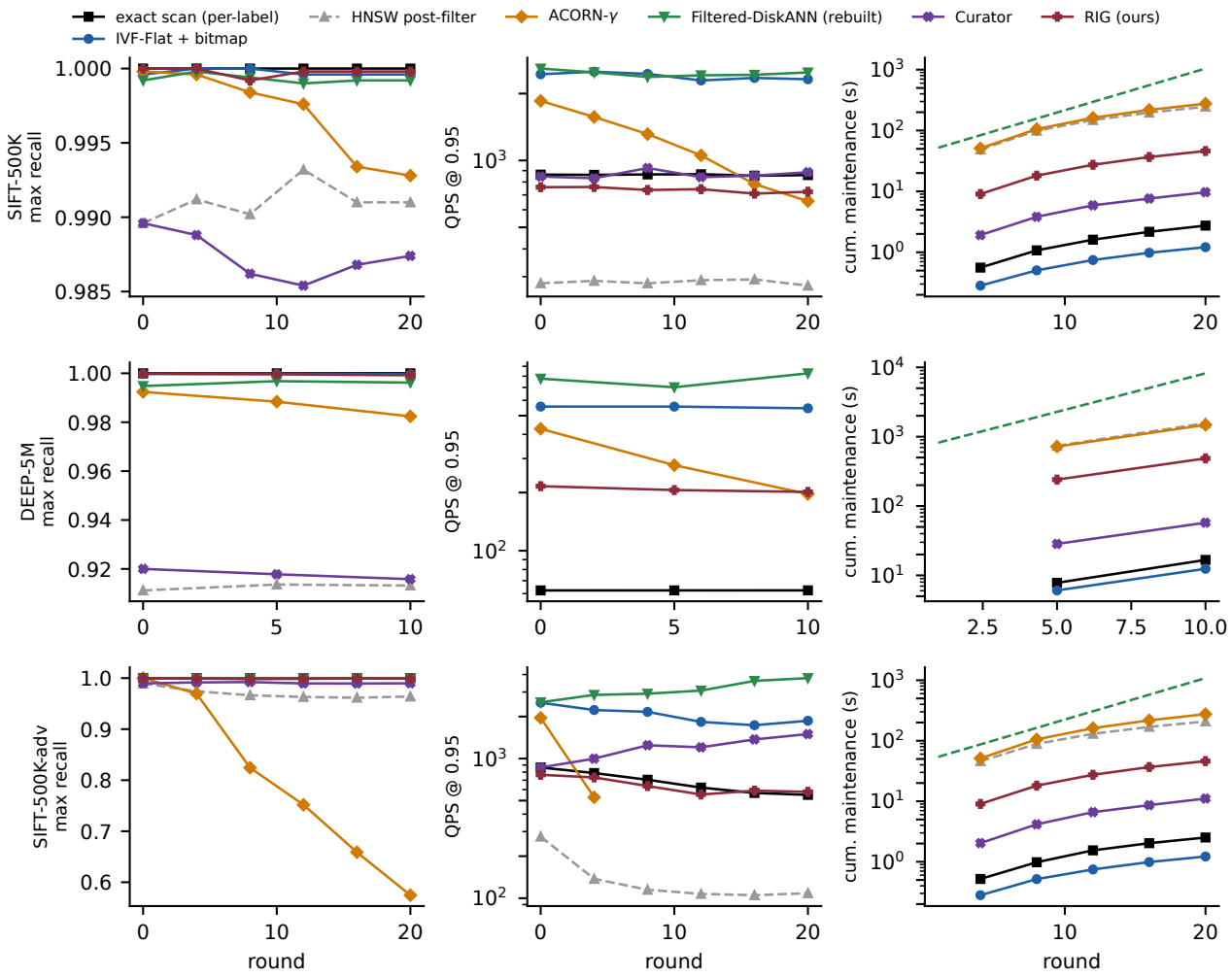


Fig. 5. Filtered search under updates (s=0.1, β=0.9). Rows: SIFT-500K, DEEP-5M, SIFT-500K-adv (Section III). Columns: maximum recall in each method's sweep, QPS at recall 0.95 (missing where not reached), and cumulative maintenance on 16 threads (Filtered-DiskANN: one rebuild per round).

VIII. Lessons for Evaluating Filtered Search (Q6)

Running ten methods from six code bases, and PostgreSQL with pgvector, on a shared machine surfaced failure modes of the *evaluation* as much as of the methods; we report them because each one distorted reported numbers before we caught it, and contention reversed a conclusion.

Machine contention. Our calibration kernel varied between 6 and 633 queries per second over the 20 days of our measurements, while repetitions inside a job varied by a few percent, so repetitions and confidence intervals do not protect a comparison whose arms ran at different times. In one early pass the scan and IVF arms of seven SIFT workloads ran at a quarter of the kernel's quiet value; re-run on a quiet machine they were up to 2.9× faster (IVF at recall 0.99 on SIFT1M, s=0.01, β=0.9: 242 to 696 QPS), enough to reverse which method wins at $s \cdot n = 10^5$. Hence the protocol of Section III: one socket, one job at a time, a gate on the calibration kernel, and the better of two passes.

Defects in baseline implementations. ACORN's incremental `add` crashed our first stream run at the first insertion round: when an upper-level neighbor list is full, the original code writes the merged candidate set back without truncating it, one entry past the node's range, corrupting the adjacency array (bulk construction takes the same path). We truncate to capacity, as upstream Faiss HNSW does, for every ACORN measurement. ACORN's documented build compiles *scalar* distance kernels: its CMake script declares the optimization level with `option()`, which turns the intended string into a boolean, so the vectorized Faiss kernels are never selected. Rebuilt with `-DFAISS_OPT_LEVEL=avx2`, ACORN built 1.5× and searched 1.1–1.4× faster on SIFT1M at identical recall; we use this build throughout. `diskannpy` pads results with *uninitialized* identifiers when a filtered search finds fewer than k matches; counting them, or even filtering them by identifier range, inflates or deflates recall depending on memory contents, so every returned identifier must be validated against the predicate.

A documented Filtered-DiskANN limitation. With one label per point Filtered-DiskANN reaches recall ≈ 0.99 at s=0.1 but stays below 0.2 at s=0.01 on SIFT1M and DEEP-10M, and larger search lists barely help (with β=0.9 at most 0.168 on SIFT1M and 0.174 on DEEP-10M): the search exhausts the part of the label's subgraph reachable from its medoid. The same behavior is reported for the original C++ tools [24] and in recent benchmarks [20], [13]; we report it as a property of the public implementation rather than of the algorithm.

Duplicate vectors. Re-inserting SIFT1M vectors under fresh identifiers charged a method a miss for returning a live copy of its target; our streams use 1.5×10^6 distinct BIGANN descriptors.

Generated numbers. Every number from the final results is a macro generated from the analysis output; the few quoted from superseded runs are marked as such.

Interpolation decides close comparisons. QPS at a recall target is interpolated between the two measured points that bracket the target, and with the doubling grids common in ANN benchmarks those points differ by a median factor of 1.69 in QPS for IVF (3.29 for post-filtering's k' grid). In 30 of our 177 index-versus-scan comparisons, the verdict changes between the lower and the upper bracketing point—among them IVF's tie with the scan at n=10^7 and $s \cdot n$=10^5. Re-measured with knob steps of $\sqrt{2}$, 20 of the 23 verdicts that the dense sweeps cover (the other 7 involve RIG or ACORN-1) stand and 3 reverse, 2 of them at ratios within 5% of one and ACORN's on SIFT1M (β=0.9, s=0.1, recall 0.99) from 1.12 to 0.911, and 12 still change within the finer bracket; claims that rest on a factor below the bracket should be measured, not interpolated. Best-of-two selection favors no family: over the mean of the two passes it raises QPS by a median 1.03× for the scan and 1.01–1.02× for the graphs.

Simple baselines. The exact scan (fastest at $s \cdot n \leq 10^4$), IVF with a bitmap (competitive static, cheapest to maintain on every stream), and a dispatcher (needed for recall 0.99 on YFCC) each changed a conclusion here; evaluations should report all three.

IX. Limitations and Threats to Validity

Measurement. All results come from one shared machine. The gating protocol of Section III removes runs taken under gross contention, but part of the first pass predates the gate, and differences below the pass-to-pass spread (90th percentile 1.10×) should not be over-read; this includes the closest 28-thread comparisons (Section V-F), such as ACORN versus IVF on DEEP-10M with β=0. The machine's single-thread speed also varied between sessions (the calibration kernel's quiet value fell by about a third in the last session, plausibly through reduced turbo headroom); every ratio we report is therefore formed within one session, and throughputs from different sessions are compared only by their ranking. Each workload has 500 queries; the standard error of mean recall, computed from the per-query recalls, has a median of 0.0017 and a maximum of 0.009 for configurations at recall ≥ 0.95, and reaches 0.022 for the low-recall YFCC configurations.

Tuning on the reported queries. Parameter sweeps and the dispatcher's threshold are evaluated on the queries whose results we report, as is customary for ANN benchmarks; applied unchanged to 500 disjoint YFCC queries, the dispatcher's choices reach a recall of 0.984–0.994 at the 0.99 target. Reported operating points are therefore optimistic for every method alike.

Workloads. The synthetic labels are quantiles of one random projection, one label per point, with correlation controlled by a single parameter; real predicates come from two datasets (YFCC-10M's tags and arXiv's categories); all queries ask for k=10. Insertions in our streams follow the distribution of the base data, so IVF's centroids, fixed after the initial training, were never stressed by distribution drift; streams with drifting insertions could reverse the update-side conclusions for IVF. Deletions are either uniform or concentrated along one direction.

What is not timed. Predicate evaluation is excluded for every method. With our implementation—intersecting sorted posting lists with numpy, materializing the 10^7-entry mask, and packing it into a bitmap—it costs 3.2 ms per YFCC query, about as much as the dispatched IVF search (99%); charged to every query of the oracle-dispatched configurations it would lower their throughput to 50–69%. A bitmap index on the tags removes almost all of it: in a separate run, where the numpy path took 2.69 ms per query, an AND of compressed Roaring bitmaps took 0.026 ms on average (0.266 ms including materializing the admissible ids), and an AND of precomputed dense bitmaps 0.063 ms, though dense bitmaps for all of YFCC's tags would not fit in memory. The predicate's cost is therefore a property of the attribute index rather than of filtered vector search, and we report it separately.

Implementations and coverage. We evaluate public implementations through their native or Python interfaces. Filtered-DiskANN is evaluated through `diskannpy`, which exposes neither a dynamic filtered index nor parallel filtered search, and its low-specificity failure is a property of that implementation; the C++ library may offer more. ACORN is evaluated with our overflow patch and an AVX2 build, with γ=12 except for one γ=100 test. We do not evaluate range-filter structures, label-centric indexes without an update path (UNG, NHQ, CAPS, SIEVE), the dynamic filtered index EMA, or vector database systems other than PostgreSQL with pgvector and Milvus; Curator is evaluated with its authors' YFCC parameters and the update path its released code provides; disk-resident indexes and concurrent query–update execution are out of scope.

Explanations and scope of claims. The cost model's terms are measured for IVF and ACORN (Section V-A); for the other methods the crossover law rests on fitted exponents over four collection sizes from one dataset (on arXiv's three sizes IVF's exponent recurs and post-filtering's does not), and Proposition 1 is a bound under independent labels, not a prediction of m^*. ACORN's collapse under correlated deletions is attributed to its update path by a fresh-build control

(Section VI). The study began as the evaluation of a new index, and its first questions were formulated after that evaluation showed simple baselines to be competitive; the hypotheses of the follow-up campaign (the crossover law, the percolation account, the rebuild period, and the robustness checks) were stated, with their falsification criteria, before it was run, and we report the ones that failed alongside the others. In particular, we had operationalized Proposition 1 as an exponent of at least 0.5; that criterion holds for IVF (interval 0.47–0.52) and for ACORN (point estimate 0.64) and fails for post-filtering (0.41), whose m^* lies far above the bound and grows more slowly, so what the data support is the bound and the sign of the exponent, not a $\sqrt{n}$ law. Several checks were added after first results and are not confirmatory: the entry-point test and the all-level repair of Section VI (after the level-0 repair left labels unrecovered), the dense sweeps (after the interpolation analysis), and the PostgreSQL, Milvus, and arXiv experiments (arXiv's lower crossover was predicted before they ran). The predicted effects of cardinality-estimation error and of best-of-two selection were not found.

Conflict of interest. RIG is the author's own, unpublished design. It was developed on SIFT1M and DEEP with the same query sets used here, which may favor it slightly; we include it as a design point, report its losses throughout, and draw no conclusion that depends on it.

X. Related Work

Experimental studies of filtered search. The closest studies evaluate filtered search on static data. Zhu et al. [11] evaluate fifteen hybrid-query algorithms with standardized attribute and range sets, varying distributions and platforms, up to 10^8 points; dynamic insertion appears in their taxonomy as a capability, but their experiments, like the others below, use static data and do not control attribute–vector correlation. Li et al. [12] compare twelve attribute-filtering methods on up to 10^7 vectors, mostly with uniform synthetic attributes, and note that their methods rarely support updates. Shi et al. [13] benchmark ten label-filter methods on six real datasets with unified tuning, including pre-filter brute force and post-filtered IVF-PQ; Iff et al. [14] contribute transformer embeddings with eleven attributes. Inside a PostgreSQL-compatible engine, Lu et al. [15] vary selectivity and correlation and find that system overheads and filter checks change which method wins; Amanbayev et al. [23] compare Faiss, Milvus, and pgvector and find IVF-Flat ahead of HNSW at low selectivity; Lim et al. [25] argue that selectivity and correlation are unstable proxies for hardness; Sanca and Ailamaki [26] characterize scan-versus-index access paths; and the Big-ANN'23 filtered track [16] already switched between brute force and an index by tag cardinality. Chronis et al. [27] survey the area; Mandarapu and Kunkunuru model plan choice as a phase transition [22], and FAVOR routes queries by estimated selectivity [28], both on static data. We do not claim the first scan or IVF baseline, nor the first controlled correlation; our contributions are the joint treatment of filters and updates and the crossover in absolute admissible-set size across n and under churn.

Streaming ANN. The Big-ANN'23 streaming track evaluates insert/search/delete runbooks without filters, and CANDY [17] benchmarks continuous ingestion. Index papers evaluate their own update paths: FreshDiskANN [8], SPFresh [9], in-place DiskANN updates [10], and incremental IVF maintenance [29]; Xiao et al. [30] document unreachable points in HNSW under replaced updates, and Yamashita et al. [31] formalize how deletion in graph indexes should be evaluated. MERIT repairs graph indexes in place on deletion, without filters [32]; Mandarapu and Kunkunuru find signal-triggered repair no better than scheduled repair at a matched budget [33]; and Das et al. separate staleness from filtered navigability in per-tenant recall under concurrent updates [34]. Dynamic *filtered* indexes exist—DIGRA [35], the dynamic range-filter index of Zhang et al. [36], DSG [37], WoW [38], Curator [20] (which we evaluate), and EMA [39]—but, as far as their papers report, each is evaluated mainly on its own update path, against static baselines or on update latency; we know of no study that compares several filtered methods, together with exact-scan and IVF baselines, while the data churns, or that varies how deletions correlate with the attribute.

Filtered-search methods not evaluated here. Range-filter structures (SeRF [5], iRangeGraph [6], UNIFY [7], window filters [40]) target range predicates; label-centric methods include NHQ [4], UNG [3], CAPS [41], and SIEVE [42]; NaviX [43] and Compass [44] target general predicates inside database systems. We restrict the study to methods with an update path and a public implementation for label or arbitrary predicates.

XI. Conclusion

Our measurements suggest four guidelines. *(i)* Keep an exact pre-filter scan over label-partitioned data, and set the dispatch threshold for the collection size and thread count at hand, not as a selectivity: for predicate-agnostic indexes the crossover grows with n. Where the predicate is a build-time label, a label-partitioned index (a per-label IVF, or Curator for the selective side) removes the crossover's growth altogether. *(ii)* Measure both correlations—attributes with embeddings, and queries with their filters—before choosing between post-filtering and a filter-aware index; with independent attributes, or with queries near their labels, post-filtering was competitive. *(iii)* Under churn, prefer indexes that delete exactly; an in-filter graph that deletes only through its predicate needs a local repair of its neighbor lists on every level, which on our correlated-deletion stream cost 1.10–1.15$\times$ the maintenance and kept recall at a fresh build's level. *(iv)* Rebuild a label-specialized graph only above the break-even ratio of queries to updates, and with a delta between rebuilds choose the period by the square-root law. For researchers, the open problems are deletion paths for in-filter graphs that repair the subgraph search depends on, and dispatchers that adapt their threshold to n, to threads, and to the update stream. All artifacts, with per-query results, are at https://github.com/suanlab/filtered-ann-updates-bench.

ACKNOWLEDGMENT

Use of AI-generated content. In accordance with the IEEE policy on AI-generated content, we disclose that an AI system, Claude Opus 5.5 (Anthropic), used through the Claude Code agent, was used substantially in this work. It implemented the harness, workload generators, baseline drivers and patches, and RIG (Sections II–III); ran and analyzed the experiments (Sections V–VII); generated every figure and table from the raw results; it identified the implementation defects reported in Section VIII; at the author's request it proposed the cost model of Section IV, the hypotheses of the follow-up campaign with their falsification criteria, and the experiments that test them, which the author approved before they were run; it drafted the text of all sections; and AI agents performed internal reviews and an audit of the manuscript, code, and results. The human author defined the research questions, made the design and scope decisions, reviewed every result and revision, and takes full responsibility for the content. No AI system is listed as an author.

REFERENCES

[1] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," *Proc. ACM Manag. Data*, vol. 2, no. 3, pp. 1–27, 2024, doi: 10.1145/3654923.

[2] S. Gollapudi et al., "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf. (WWW)*, 2023, pp. 3406–3416, doi: 10.1145/3543507.3583552.

[3] Y. Cai, J. Shi, Y. Chen, and W. Zheng, "Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 1–27, 2024, doi: 10.1145/3698822.

[4] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "An efficient and robust framework for approximate nearest neighbor search with attribute constraint," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 15738–15751, doi: 10.52202/075280-0692.

[5] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, "SeRF: Segment graph for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 1–26, 2024, doi: 10.1145/3639324.

[6] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, "iRangeGraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 1–26, 2024, doi: 10.1145/3698814.

[7] A. Liang, P. Zhang, B. Yao, Z. Chen, Y. Song, and G. Cheng, "UNIFY: Unified index for range filtered approximate nearest neighbors search," *Proc. VLDB Endow.*, vol. 18, no. 4, pp. 1118–1130, 2024, doi: 10.14778/3717755.3717770.

[8] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, "FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search," arXiv:2105.09613, 2021.

[9] Y. Xu et al., "SPFresh: Incremental in-place update for billion-scale vector search," in *Proc. 29th ACM Symp. Operating Syst. Princ. (SOSP)*, 2023, pp. 545–561, doi: 10.1145/3600006.3613166.

[10] H. Xu, M. D. Manohar, P. A. Bernstein, B. Chandramouli, R. Wen, and H. V. Simhadri, "In-place updates of a graph index for streaming approximate nearest neighbor search," arXiv:2502.13826, 2025.

[11] J. Zhu et al., "An experimental evaluation of hybrid querying on vectors," *Proc. VLDB Endow.*, vol. 19, no. 2, pp. 183–195, 2025, doi: 10.14778/3773749.3773757.

[12] M. Li, X. Yan, B. Lu, Y. Zhang, J. Cheng, and C. Ma, "Attribute filtering in approximate nearest neighbor search: An in-depth experimental study," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–26, 2025, doi: 10.1145/3769763.

[13] J. Shi, Y. Cai, and W. Zheng, "Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study," arXiv:2509.07789, 2025.

[14] P. Iff, P. Brügger, M. Chrapek, D. Kochergin, M. Besta, and T. Hoefler, "Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors," in *Proc. 49th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, 2026, pp. 610–622, doi: 10.1145/3805712.3809731.

[15] D. Lu et al., "An in-depth study of filter-agnostic vector search on a PostgreSQL database system," *Proc. ACM Manag. Data*, vol. 4, no. 3, pp. 1–26, 2026, doi: 10.1145/3802011.

[16] H. V. Simhadri et al., "Results of the Big ANN: NeurIPS'23 competition," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025, pp. 76394–76408, doi: 10.52202/085713-2303.

[17] X. Zeng, Z. Wu, X. Hu, X. Shi, S. Sun, and S. Zhang, "CANDY: A benchmark for continuous approximate nearest neighbor search with dynamic data ingestion," arXiv:2406.19651, 2024.

[18] J. Wang et al., "Milvus: A purpose-built vector data management system," in *Proc. Int. Conf. Manage. Data (SIGMOD)*, 2021, pp. 2614–2627, doi: 10.1145/3448016.3457550.

[19] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[20] Y. Jin et al., "Curator: Efficient vector search with low-selectivity filters," *Proc. ACM Manag. Data*, vol. 4, no. 1, pp. 1–27, 2026, doi: 10.1145/3786635.

[21] M. Douze et al., "The Faiss library," *IEEE Trans. Big Data*, vol. 12, no. 2, pp. 346–361, 2026, doi: 10.1109/TBDATA.2025.3618474.

[22] M. Mandarapu and S. Kunkunuru, "Filtered ANN as a phase transition: When selectivity-estimation error causes plan regret," arXiv:2606.16341, 2026.

[23] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered approximate nearest neighbor search in vector databases: System design and performance analysis," arXiv:2602.11443, 2026.

[24] Microsoft DiskANN GitHub issue #550, "[Question] Low Recall in Filtered-Disk ANN on Sift Dataset-1M with Uniform and Zipf Distributions," https://github.com/microsoft/DiskANN/issues/550, accessed 2026.

[25] M. Lim, D. Kim, M. Kim, and J. Do, "Revisiting filtered ANN benchmarks: A hardness-controlled benchmark generator for realistic evaluation," *Proc. VLDB Endow.*, vol. 19, no. 10, pp. 2763–2776, 2026, doi: 10.14778/3828612.3828630.

[26] V. Sanca and A. Ailamaki, "Efficient data access paths for mixed vector-relational search," in *Proc. 20th Int. Workshop Data Manage. New Hardware (DaMoN)*, 2024, pp. 1–9, doi: 10.1145/3662010.3663448.

[27] Y. Chronis, H. Caminal, Y. Papakonstantinou, F. Özcan, and A. Ailamaki, "Filtered vector search: State-of-the-art and research opportunities," *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5488–5492, 2025, doi: 10.14778/3750601.3750700.

[28] J. Song et al., "FAVOR: Efficient filter-agnostic vector ANNS based on selectivity-aware exclusion distances," arXiv:2605.07770, 2026.

[29] J. Mohoney et al., "Incremental IVF index maintenance for streaming vector search," arXiv:2411.00970, 2024.

[30] W. Xiao, Y. Zhan, R. Xi, M. Hou, and J. Liao, "Enhancing HNSW index for real-time updates: Addressing unreachable points and performance degradation," arXiv:2407.07871, 2024.

[31] T. Yamashita, D. Amagata, and Y. Matsui, "How should we evaluate data deletion in graph-based ANN indexes?" presented at the *NeurIPS 2025 Workshop on Machine Learning for Systems*, 2025, arXiv:2512.06200.

[32] Z. Wu et al., "MERIT: Efficient in-place deletion for dynamic graph-based approximate nearest neighbor indexes," arXiv:2607.29173, 2026.

[33] M. Mandarapu and S. Kunkunuru, "When to repair a graph ANN index: A matched-budget negative result, and the interpolated-baseline trap that hid it," arXiv:2607.00728, 2026.

[34] N. Das, K. Devadkar, and A. Godbole, "Recall-bounded staleness for filtered multi-tenant ANN graph indexes under concurrent updates," arXiv:2610.15130, 2026.

[35] M. Jiang et al., "DIGRA: A dynamic graph indexing for approximate nearest neighbor search with range filter," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 1–26, 2025, doi: 10.1145/3725399.

[36] F. Zhang et al., "Efficient dynamic indexing for range filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 1–26, 2025, doi: 10.1145/3725401.

[37] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, "Dynamic range-filtering approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 10, pp. 3256–3268, 2025, doi: 10.14778/3748191.3748193.

[38] Z. Wang, J. Zhang, and W. Hu, "WoW: A window-to-window incremental index for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–27, 2025, doi: 10.1145/3769843.
[39] M. Li, B. Lu, J. Cheng, and C. Ma, "EMA: Approximate nearest neighbor search with general attribute filtering and dynamic updates," arXiv:2606.00734, 2026.
[40] J. Engels, B. Landrum, S. Yu, L. Dhulipala, and J. Shun, "Approximate nearest neighbor search with window filters," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 12469–12490.
[41] G. Gupta, J. Yi, B. Coleman, C. Luo, V. Lakshman, and A. Shrivastava, "CAPS: A practical partition index for filtered similarity search," arXiv:2308.15014, 2023.
[42] Z. Li, S. Huang, W. Ding, Y. Park, and J. Chen, "SIEVE: Effective filtered vector search with collection of indexes," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4723–4736, 2025, doi: 10.14778/3749646.3749725.
[43] G. Sehgal and S. Salihoğlu, "NaviX: A native vector index design for graph DBMSs with robust predicate-agnostic search performance," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4438–4450, 2025, doi: 10.14778/3749646.3749704.
[44] C. Ye, X. Yan, and E. Lo, "Compass: General filtered search across vector and structured data," arXiv:2510.27141, 2025.